

2400-DS1AL # Algorithms for Data Science

[Kokpit](#) / [Moje kursy](#) / [2400-DS1AL#2025L#277553](#) / [Lecture 10 & 11](#) / [Lecture notes](#)

Lecture notes

Credits: dr Krzysztof Fleszar

Lecture 7: Dictionaries

Recall that data structures are ways of arranging data. In most data structures there is some important property that defines how data is arranged. This property has to be satisfied after every operation (this property is called the invariant of the structure—similarly to loop invariants, which must be satisfied after every iteration of the loop). So far we have seen the following data structures:

- arrays, which can be unsorted (no invariant) or sorted (the invariant here is that the array should be sorted after every operation). They can be of fixed size (a continuous block of memory that cannot grow) or they can be dynamic (we can append elements to it, like in the Python list)
- complete binary heap (as used in HeapSort)

In an unsorted array, also called unordered array, we can add a new element very quickly, but it takes very long to find anything. In a sorted array, we can find every given element very quickly (the invariant helps us); however, as we have seen in insertion sort, adding a new element takes time (we need to spend extra time to keep the invariant satisfied). This is a common tradeoff: it takes more time to make the data more ordered, but then it takes less time to search!

In this lecture, we will explain amortized analysis (a useful technique of computing the total time complexity of a sequence of operations on a data structure), and show some data structures that can be used as dictionaries.

Amortized Analysis

Algorithms that are based on data structures typically perform a sequence of operations on these data structures. While some of these operations take a comparatively long time to perform, the data structures are designed in such a way that such slow calls are very rare, and the total running time is good. Amortized analysis is a tool to deal with such situations. The idea is similar to that of amortization in bookkeeping, hence the name.

Idea: We assign amortized time complexity to each operation on our data structure in such a way that, when we consider a sequence of operations, the actual total time complexity of this sequence will never exceed the sum of the given amortized time complexities, even though the actual running time of an individual operation might exceed its amortized time. In other words, if the amortized running time is $O(\alpha(n))$ for some operation (where n is an upper bound on the number of elements in the data structure), then the total worst-case running time of a sequence of k such operations is bounded by $O(k\alpha(n))$.

Example: according to [TimeComplexity on the Python wiki](#), "amortized worst case" complexity of appending an item to a Python list is $O(1)$. This means that, if you perform k append operations, they will take in worst case $O(k)$ time in total. However, it is possible that an individual append operation will take more than $O(1)$ time.

A useful method of computing the amortized time complexity is the **potential method**. *This method is out of scope of our course, however, in the following I briefly explain it to the interested reader.* After each operation, our data structure has the *potential* $\Phi(S)$, which is a nonnegative number depending on its state S . (Defining a suitable potential function is a nontrivial task.) If an operation op changes the state from S to S' , we calculate the amortized time complexity of this operation as $a(op) = t(op) + \Phi(S') - \Phi(S)$, where $t(op)$ is the actual time. Intuitively—the current potential is our savings account, and $a(op)$ is the number of credits we are given to perform the operation op . These credits can be either used right away to pay for the computations (the $t(op)$ part), or we could use our savings account (the $\Phi(S') - \Phi(S)$ part), either by storing unused credits into the account, or by using our savings to cover the extra costs when $t(op) > a(op)$. The potential starts at 0, so this agrees with the idea given above—if we perform a sequence of operations $op_1, \dots, op_k$, we obtain $a(op_1) + \dots + a(op_k)$ credits in total, and every unit of time used by each opi had to be paid for, either from $a(op_i)$ or from the savings from an earlier operation.

Example: Dynamic Array

We have already mentioned that Python lists and C++ vectors store their values in a block of consecutive cells in the memory. Thus, for example, a vector with n elements will have them stored in memory cells numbered $k, \dots, k+(n-1)$, for some k . But what to do when we want to append a new element to our list? The memory cell $k+n$ could be already used for something else!

To solve this problem we have to allocate a new block of memory where all the values will fit. If we allocate a new block of size $n+1$, we will need to move all the n old values to it, which will require time $\Theta(n)$. Not a good choice—potentially we will have to perform such a move whenever we append a new element. Appending a single element in time $\Theta(n)$ is not very efficient.

A better solution is to reserve space for the new elements which will potentially come in the future. We allocate a new block of size $2n$ instead of $n+1$. This way, the next $n-1$ append operations will be very fast; each operation will take only $O(1)$ time. Then, when adding yet another element, we will have to move all the elements again—an expensive operation, but the next $2n-1$ operations will be fast again. An array growing in this way is usually called a dynamic array; this technique is used both in Python lists and C++ vectors.

As we can see, most of the operations take time $O(1)$, but occasionally very expensive operations will occur. We can use the potential method to show that the amortized time complexity of the append operation is $O(1)$. The following proof is out of scope of our course, but I mention it for the interested reader. Let the potential $\Phi(S)$ be $2n-m$, where n is the number of elements in the dynamic array, while m is the current total size of the array including the reserved space.

- As long as the reserved space is not yet full, we use $O(1)$ time to put the appended value in the next memory cell, and the potential increases by 2—the total amortized cost is $O(1)$.
- When the reserved space becomes full ($m=n$), we allocate a new array of size $2m$, and move all the n elements there—the actual cost is $\Theta(n)$. However, the potential drops from n to 0, which is enough to cover the cost of this operation.

Conclusion

For most applications, we can simply ignore the difference between the amortized and actual worst-case time. For example, if we design an algorithm that performs multiple append operations on Python lists, it will be alright to assume that this operation simply takes time $O(1)$ —even if on some rare occurrences the operation takes longer, it is not important at all since the running time will be "amortized". One situation where the difference matters is when we want to respond to user input in real time—if some of their queries take longer than expected (e.g., because we needed to resize our lists at that particular time), they will notice it, even though the average time is alright!

Dictionaries

A dictionary (aka associative array) is a data structure D which is basically a set of key-value pairs (where keys do not repeat), with the following basic operations:

- $D = \text{Init}()$ —create a new dictionary,
- $D.\text{Add}(k, v)$ (aka Insert)—insert the new key-value pair (k, v) (or, if a key-value pair with key k already exists in D , we update its value to the new value v instead of inserting a new key-value pair),
- $D.\text{Check}(k)$ —check if D contains a key-value pair with the key k ,
- $D.\text{Get}(k)$ (aka Lookup)—get the value associated to the key k in D .

These are the basic operations—in the last lecture (Dynamic Programming), we have shown how to use a dictionary for an algorithm based on memoization, and we used all of these operations. In some cases, more operations might be useful—e.g. $D.\text{Remove}(k)$, which removes the key-value pair with the key k from the dictionary. We will focus on the ones above, though. We only note that, in dictionaries, the remove operation has normally the same running time as the add operation. The following part of the lecture will give possible implementations of a dictionary.

Click [here](#) for a visualization of Solutions (2) to (6).

Solution (1): An Array Indexed by Keys

Suppose that our keys are $0, \dots, M-1$. We simply use an array a (e.g., a C++ vector or a Python list) of length M . If the key k is currently not in the dictionary, then $a[k]$ is None; otherwise, $a[k]$ is the value associated with k . What are the time and memory complexities of each operation?

- Memory complexity is $O(M)$.
- Init takes $O(M)$ time.
- Add, Check, and Get take $O(1)$ time.

This solution is practical only when M is small. Otherwise, although Add, Check and Get are all very fast, our structure takes a lot of memory, and also a lot of time to initialize.

Solution (2): An Unsorted Array of Key-Value Pairs

In this solution, we simply store an unordered list of key-value pairs in our (dynamic) array. For example, a dictionary with value 10 at key 10, and value 30 at key 20, could be stored as $[(10,10), (20,30)]$, or also as $[(20,30), (10,10)]$. What are the complexities?

- Memory complexity is $O(n)$ where n is the number of key-value pairs.
- Init takes $O(1)$.
- Add takes $\Theta(n)$ in the worst case. However, the amortized time is $O(1)$ if we assume that the key is not yet in the dictionary—we simply append the element to the end.
- To perform Check and Get, we need to go through the whole array, which takes time $O(n)$.

Thus, although Add is very fast (assuming that the key is not yet present), the Check and Get operations are very slow. Typically, we perform a large number of operations of every type (Add and Check/Get), so we would like both operation types to be fast—if one is extremely fast and the other one is slow, it is not sufficient for us.

Solution (3): A Sorted Array of Key-Value Pairs

Could we reduce the time of Check/Get to $O(\log n)$ by using binary search? Not in the last solution—since the Add operation always appends new elements to the end of our list, the array is not likely to be sorted. But a sorted array is a prerequisite for binary search. Thus, what if we change the Add operation such that our array is guaranteed to be always sorted?

- Memory complexity is $O(n)$ where n is the number of key-value pairs.
- Init takes $O(1)$.
- Check and Get work in $O(\log n)$, using binary search.
- Unfortunately, Add is slow. Although we can replace the value (once we know where it is after $O(\log n)$ time) in time $O(1)$, or append a new value to the end in (amortized) time $O(1)$, it is difficult to insert a new element somewhere in the middle—we need to push all larger elements to the right. Therefore, Add needs time $O(n)$.

We could use a combination of (2) in (3) in the cases where we first add all the elements to the dictionary, and only then perform only lookups: we append the elements one by one to the end ($O(1)$ time per append), then we sort the whole array (in $O(n \log n)$), and finally use quick lookups using binary search ($O(\log n)$). However, if we interleave lookups and insertions, we will need to use a better solution.

This is a common tradeoff in algorithmics (and also in real life)—if our data is arranged in an orderly way, it is easy to search for something, however, we have to spend some time to make it ordered. Thus, we need to find the correct balance between chaos and order. Neither solutions (2) nor (3) did achieve the correct balance.

Solution (4): A Linked List

The problem in solution (3) was that it is difficult to insert an element in the middle of a list. A linked list (commonly called just list—do not confuse it with the Python list, which is a dynamic array rather than a linked list) is a data structure which allows quick insertions.

We could insert the elements in our arrays as triples: (key, value, next), where next is the index of the next element (it "links" to the next element); the last element points to the special value None. This allows us to insert new elements quickly. For example, suppose we have: [(10, 1, 2), (30, 9, 3), (20, 4, 1), (50, 25, None)]. Let $a[0]$ be the first element—thus, the first element (10, 1, 2) has the key 10 and the value 1 and it points to position 2; therefore, the next element in our list is $a[2]$, that is, (20, 4, 1); then, we have $a[1]$, that is, (30, 9, 3); then $a[3]$, that is, (50, 25, None); and this is the last element.

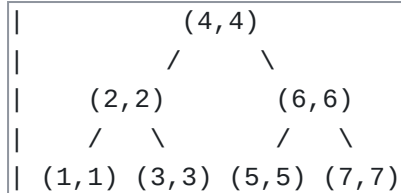
Suppose we want to insert a new key-value pair (40,16) between $a[1]=(30,9,3)$ and $a[a[1].next] = a[3] = (50,25,None)$ —to do this, we append (40,16,3) as $a[4]$, and we change $a[1].next$ to 4. Thus, $a[1] = (30,9, 4)$.

We can use this method to add elements in time $O(1)$, but only if we know the position of its predecessor. However, our sorted list is now arranged in the memory rather chaotically—and searching will again take $O(n)$. Hence, linked lists are not useful as dictionaries (however, they are useful for other purposes).

Note: In the (key, value, next) representation, next is a index in the table—in most programming language, you would rather put a 'pointer' or 'reference' to the next element instead. In Python, this would look like: [10,1,[20,4,[30,9,[50,25,None]]]] (Python treats lists as references). However, the low level representation is similar to the (key, value, next) representation—except that next is an actual address of a memory cell, rather than an array index. Also note that for many applications, it is useful if we also have a link to the previous element. Such lists are called doubly linked lists.

Solution (5): A Binary Search Tree (BST)

How to combine some of the ideas in Solution (4) with binary search? Instead of arranging our items in a *linked list*, we arrange them into a *tree*. Just as every item in a linked list had a link to the 'next' element, every element in a tree will have a link to its 'left' and 'right' child. (For most applications it is also useful to have a link to the parent.) For example, here is a tree containing (k, k) for $k = 1..7$:



The invariant of BST is that every element to the left from the root will have a smaller key than the root, and every element to the right will have a greater key. The same rule holds for the subtrees - for example, every element in the right subtree of (2,2) will have a key greater than 2 (but smaller than 4). In other words, for any element it holds that all the keys in its left subtree are smaller and all keys in its right subtree are larger than the key of the element.

We can perform lookups by going down our tree—we always know where to go. We can also do insertions easily—we simply insert the new element in the location where we were looking for and found nothing.

BST Invariant: the key of a node is larger than all the keys in its left subtree, and smaller than all the keys in its right subtree.

The BST above could be created by inserting elements e.g. in the following order: 4, 6, 2, 3, 7, 5, 1. Using the (key, value, left, right) representation, we have:

- After inserting (4,4): [(4,4,None,None)]
- After inserting (6,6): [(4,4,None,1), (6,6,None,None)]
- After inserting (2,2): [(4,4,2,1), (6,6,None,None), (2,2,None,None)]
- After inserting (3,3): [(4,4,2,1), (6,6,None,None), (2,2,None,3), (3,3,None,None)]
- And so on...

What is the complexity?

- Memory complexity is again $\Theta(n)$.
- Init takes $O(1)$.
- Insert and lookup take time proportional to the length of the search path.

A BST is quite similar to the simplest implementation of QuickSort—it works very nicely if we insert elements randomly, as the expected path length is $O(\log n)$ in this case; however, if we insert them from smallest to greatest key (or, from greatest key to smallest key), the tree will basically degenerate into a list, and both insertions and lookup take time $\Omega(n)$. Therefore BST is a good solution only if we know that elements are inserted in a somewhat random order.

Solution (6): A Self-Balancing Binary Search Tree

As mentioned, binary search trees had $\Omega(n)$ worst case time complexity. What if we could detect this worst case, and rebuild the tree if this happens, so that we always obtain paths of length $O(\log n)$? Self-balanced trees are based on this idea.

There are many efficient self-balancing binary search trees—we will sketch the AVL tree, which was historically the first, and probably is still the easiest.

But first, we need a small definition. Let $h(v)$ be the height of the subtree with root v which is the length (number of vertices) of a longest path in the subtree starting at v . For instance, if v has no children, then $h(v)=1$. Otherwise, if v has at least one child, then the height of v is larger by exactly 1 than the maximum over the heights of its children. Summarized, we can write

$$h(v) = \max(h(\ell), h(r)) + 1$$

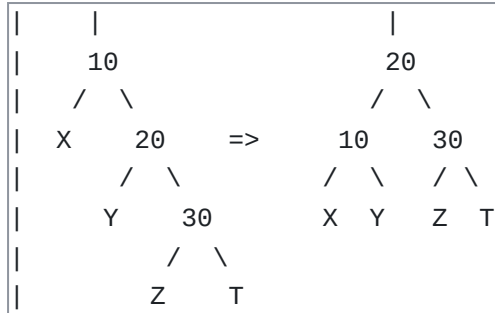
where ℓ and r are the left and right children pointers of v , and where we define $h(\text{None})=0$ (to take account of the case where ℓ or r does not exist, that is, points to None).

In an AVL tree, we store for every vertex v the height $h(v)$ of its subtree (we store $h(v)$ in addition to key, value, left and right link). We also keep the following *invariant*: for each vertex v , the difference between $h(\ell)$ and $h(r)$, where ℓ and r are the left and right children of v , is at most 1.

AVL Invariant: the left and right subtrees of a node differ at most by 1 in their height.

It can be shown that, if we have n elements in an AVL tree, its height will be $O(\log n)$. Thus, lookups can be done in $O(\log n)$ time.

When we add a new element w to an AVL tree, we need to update $h(v)$ for each vertex on the path from w to the root. If we do this like in a BST, it is possible that we break our invariant—for example, if we add the elements 10, 20, 30 in that order, then the first element (root) will have $h(10)=3$, a right son with $h(r)=2$ and no left son ($h(\ell)=0$). This can be solved with a *rotation*—we rebuild the subtree in the following way :



In this example, $X, Y, Z, T = \text{None}$, but the rotation works in the same way when these variables symbolize different subtrees. Note that for a single rotation, we just change a constant number of links; hence, a single rotation takes constant time.

We need to go from w back to the root, fixing the values of $h(v)$ on the way, and possibly performing a rotation to the left or to the right. Since this path was of length $O(\log n)$ and each rotation takes $O(1)$ time, insertion also works in $O(\log n)$ in total. To sum up:

- Memory complexity is again $\Theta(n)$.
- Init takes $O(1)$.
- Insert and lookup take time $\Theta(\log n)$ (Θ as the height of the tree is always $\Omega(\log n)$.)

(Note: the AVL trees are more commonly described not by giving $h(v)$ for each vertex v , but rather by giving the balance factor of each vertex—which corresponds to $h(r) - h(l)$. The balance factor of every vertex in a correctly balanced AVL is $-1, 0$, or $+1$; it may temporarily become -2 or $+2$ before a rotation.)

Other self-balancing binary search trees include, e.g., *red-black trees*, *B-trees*, and *splay trees*. Probably the easiest self-balancing binary search tree is a *treap* (=tree+heap)—every element gets a random priority, and elements with a high priority are rotated to the top. This works similar to a BST with randomly inserted elements. Thus, if we are unlucky, a treap might get long paths.

Solution (7): A Hash Table

Solutions (2) to (6) could be understood as a gradually refinement of the initial ideas (2) and (3). Hash tables go into another direction—they are rather based on Solution (1).

The problem with Solution (1) was that we needed as much memory as we have keys. For example, if our keys are numbers from the range 0 to 1000,000,000,000, we need to allocate an array of size 1000,000,000,000 even if the number n of key-element pairs is much smaller. However, we rarely

can afford that much memory. But what if we allocate an array with just N cells for some smaller N , for example, $N=1000$, and use the cell $h(k)=k \bmod N$ to store the information about key k ? The value of $h(k)$ is called a hash, the function h is called a hash function, and an array indexed by hash values is called a hash table.

This solution has one big problem—the possibility of a collision: if two keys k_1 and k_2 have $h(k_1)=h(k_2)$, they go to the same cell in our array. We can solve this problem in the following way: we don't put a single key-value pair in $a[i]$, but rather all the inserted key-value pairs whose hash value is i . We need a sub-dictionary to put these key-value pairs here; we could use any of the data structures from points (2) to (6) for this purpose—let's use the simplest one of them, i.e., an unsorted dynamic array.

How efficient is this? If we are lucky, we will insert roughly n/N elements into each position. If we have bad luck, all elements go into the same cell. In both cases, the complexity is:

- Memory complexity is $\Theta(N+n)$.
- Init takes $\Theta(N)$.
- Insert and lookup take time $\Theta(N+n)$ (worst case) and $O(n/N)$ ("lucky case").

But will we be lucky? If we are using the hash function $h(k)=k \bmod N$ with $N=1000$, then we are likely to get unlucky—for example, if every value inserted is a multiple of 1000, then all of them will go to the same cell. However, we should be fine if our hash function is sufficiently "random"—then we are lucky with very high probability. There is a big theory of creating good hash functions—they need to be as random as possible, and quickly computable at the same time. For example, if we select our hash function at random from a so-called universal family of hash functions and ensure that N is not too small, then with arbitrary high probability we are lucky!

What N should we pick? In Theory, it suffices that $N=\Omega(n)$, but for best results, it should be greater than n ($N>2n$ is used for example in the so-called Cuckoo hashing), but not too large. We could use the same trick as in the dynamic array: we try to keep the invariant that $N>2n$; when we insert enough elements that this relation no longer holds, we allocate a new hash table, doubling the value of N . This way, we obtain the following complexity (if we are lucky and don't get many collisions):

- Memory complexity is $\Theta(n)$ (as now $N=\Theta(n)$ and thus $\Theta(N+n)=\Theta(n)$),
- Init, insert, and lookup all take time $\Theta(N)$ (worst case) and $O(1)$ (expected case, that is, worst case with high probability) (as now $n/N=O(1)$).

Everything is as good as possible—under the assumption that we are lucky. And we are with very high probability.

Sometimes, we can even enforce our luck: if we know all the n keys in advance, then we can precompute a hash function that will have no collisions at all as long as $N > 5n$. In other words, we obtain a hard guarantee of $O(1)$ worst case running time for the respective operations.

Conclusion

This section is not obligatory but certainly interesting and helpful.

Luckily, in most modern programming languages, we don't need to implement these complicated data structures by ourselves. In C++, we have the type map for dictionaries based on red-black trees (a type of self-balancing BSTs), and `unordered_map` for dictionaries based on hash tables. While self-balancing BSTs have worse average time complexities, they have two advantages to hash tables: (i) they work well even in the worst case, (ii) they allow for some additional operations. For example, after finding a given key in the dictionary, we can easily (in $O(\log n)$) find the next/previous key in order—due to the random nature of hash values, this is impossible in a hash table (hence the name, `unordered_map`).

Python dictionaries are based on hash tables. This is because the two disadvantages mentioned above are almost never relevant in practice, and the designers of Python have decided to keep it simple. In the rare cases where we would prefer to use a self-balanced BST, we can use a third party library.

In general, there is rarely need to implement a hash table or a self-balancing BST on our own. One situation where implementing our own AVL (or another type of self-balancing BST) is useful is augmentation—we extend our AVL with additional abilities. The idea of these augmentations is that for every vertex v we also store some kind of general information about all the elements in the subtree rooted at v . For example, if for every vertex v we remember the number of the elements in the subtree rooted at v , we can get to a vertex w which is the k -th successor of v in time $O(\log k)$; the normal AVL only allows doing this in $O(k)$.

Try to call the function `hash` in Python to compute the hash value of some numbers and strings. You observe that the hash function returns a large number. However, the Python dictionary is not as large as this number—in reality, this large number is taken modulo the current size of the hash table. Python uses a different method of resolving collision than the one explained above: we put only one (key, value) pair in each cell of the hash table; in case of a conflict, we use the full hash value again to compute another position, and try to insert/search there. We repeat until we have either found the key, or reached an empty location. (Of course, this works only if the size of the hash table is greater than the number of elements in it, and is efficient only if it is significantly greater—this is the reason why we have chosen $N > 2n$.)

Some extra experiments with the Python hash function:

- Set $v = 2^{62} - 2$. For $k = 1 \dots 100000$, try to insert the keys $k*v$ into a Python dictionary. What do you observe? Repeat the same for some other value for v . To what hash does $\text{hash}(k*v)$ resolve?
- *Answer (Select to see):* If we compute $\text{hash}(k*v)$ for $v = 2^{62} - 2$ and various values of k , we always obtain 0. Since the Python dictionary is a hash table using the function $\text{hash}()$ —it will be exceptionally slow, due to the number of collisions.
- Run $\text{hash}(\dots)$ on a string and print it! Restart your program. What do you observe? Try to find out in the Internet why this happens!
- *Answer (Select to see):* In newer versions of Python, you notice that hash values of strings are different when you run Python again—this is to protect from "denial of service" attacks, where the attacker tries to overload and crash a server by sending artificially generated queries that have the exact same hash value.

Ostatnia modyfikacja: wtorek, 2 czerwca 2026, 22:21

[◀ Lecture 11 - recording](#)

Przejdź do...

[Lecture 12 ▶](#)

✉ Skontaktuj się z pomocą techniczną

Jesteś zalogowany(a) jako Ondřej Marvan (Wyloguj)
2400-DS1AL#2025L#277553